ASSIGNMENT 3

Database Design Analysis

Online Shopping Management System

1. Introduction

This report analyses the finalized OnlineShoppingDB design for data redundancy, functional dependencies, and insertion, deletion, and update anomalies. The design separates customers, products, orders, order lines, and payments to reduce repeated facts.

2. Finalized database tables

Table	Primary key	Main columns / relationship
Customers	CustomerID	CustomerName, Email (UNIQUE), Phone, Address
Products	ProductID	ProductName, Category, Price, Stock
Orders	OrderID	CustomerID references Customers; OrderDate; Status
OrderDetails	OrderDetailID	OrderID references Orders; ProductID references Products; Quantity; UnitPrice
Payments	PaymentID	OrderID references Orders; PaymentMethod; Amount; PaymentStatus

3. Functional dependencies

Table	Functional dependency	Reason
Customers	CustomerID → CustomerName, Email, Phone, Address; Email → other customer attributes	CustomerID identifies one customer. Email is unique.
Products	ProductID → ProductName, Category, Price, Stock	ProductID identifies one product record.
Orders	OrderID → CustomerID, OrderDate, Status	OrderID identifies one order.
OrderDetails	OrderDetailID → OrderID, ProductID, Quantity, UnitPrice	OrderDetailID identifies one order line.
Payments	PaymentID → OrderID, PaymentMethod, Amount, PaymentStatus	PaymentID identifies one payment record.

4. Redundancy and anomalies

Data redundancy: Customer name and address are stored once in Customers rather than repeated in every order. Product descriptions and current prices are stored in Products. OrderDetails stores UnitPrice intentionally because it is the price charged at purchase time and may differ from the current Products.Price.

Insertion anomaly: In a poorly designed combined table, a new product could not be recorded until an order existed, or customer details might need to be repeated. Separate Customers and Products tables allow records to be inserted independently.

Update anomaly: If a customer's address were repeated across many order rows, changing it in only some rows could create conflicting addresses. Here, the address is updated in Customers once.

Deletion anomaly: In a combined table, deleting the last order line for a product might accidentally remove the only stored product description. Separate Products and Orders tables avoid this problem. Historical records still need a deliberate deletion/retention policy.

5. Normalization and proposed improvements

The tables use atomic values, supporting First Normal Form (1NF). Non-key attributes depend on the key of their table, supporting the intended 2NF/3NF structure. Customer attributes are not stored in Orders and product descriptions are not stored in OrderDetails.

- Add UNIQUE (OrderID, ProductID) to OrderDetails if the same product should appear only once per order.
- Add CHECK constraints for allowed order statuses, payment methods, and payment statuses.
- Decide whether Payments should be one-to-one or one-to-many with Orders. Add UNIQUE (OrderID) if only one payment record is allowed per order.
- Document a deletion policy, such as restricting deletion of customers with historical orders.
- Consider preserving shipping/billing address snapshots if order-address history must be retained.

6. Conclusion

The five-table design reduces common redundancy and anomalies by separating entities and using foreign keys. Constraints and documented business rules can further improve consistency. UnitPrice in OrderDetails is intentional historical transaction data, not unnecessary duplication.